

VFMVAC: View-filtering-based multi-view aggregating convolution for 3D shape recognition and retrieval

Zehua Liu, Yuhe Zhang*, Jian Gao, Shurui Wang

School of Information Science and Technology, Northwest University, Xi'an, PR China

ARTICLE INFO

Article history:

Received 30 August 2021

Revised 5 April 2022

Accepted 1 May 2022

Available online 3 May 2022

Keywords:

Multi-view
Channel shuffle
Convolution
Recognition
Retrieval

ABSTRACT

Multi-view based 3D shape recognition methods have achieved state-of-the-art performance in 3D shape recognition and retrieval. The main focus of multi-view based approaches is determining how to fuse multi-view features into a compact, descriptive, and robust 3D shape descriptor that can then be utilized for 3D shape recognition and retrieval. This paper proposes a novel multi-view aggregating framework, view-filtering-based multi-view aggregating convolution (VFMVAC) to learn global shape descriptors for 3D shape recognition. The proposed VFMVAC applies a voting-based view filtering strategy to select representative views, also introduces a novel multi-view aggregating module to integrate multi-view features; this substantially improves the descriptiveness of the descriptors, and therefore improves the performance of 3D shape recognition and retrieval. Specifically, all views are fed into a voting-based view filtering module to select the top-k representative views. Subsequently, the features of the top-k views are fed into the multi-view aggregating module, which first conducts cross-view channel shuffle for achieving cross-view information flowing, and the resulted reshaped features are then fed into the aggregating convolution module for feature fusion. Experiments on benchmark datasets demonstrate that the proposed VFMVAC is effective and outperforms several recent techniques with respect to the classification and retrieval performance, robustness and efficiency.

© 2022 Elsevier Ltd. All rights reserved.

1. Introduction

With the rapid development of three-dimensional (3D) acquisition technologies and devices, 3D models have been employed in various applications, such as the virtual/augmented reality [1] and autonomous driving [2]. 3D models, which are usually represented in different formats, such as point clouds, volumetric meshes, and two-dimensional (2D) images captured from multiple viewpoints (multi-view images), are virtual renderings of natural objects. Thus, 3D model recognition has become a fundamental and essential research topic in the fields of pattern recognition and computer vision.

In recent years, many academics have studied the problem of 3D model recognition, the related research methods can be roughly classified into three main categories, namely point-based, voxel-based, and multi-view based methods. Point-based methods [3–7] directly input all points into the network for feature learning, making it difficult to tackle the issue of data irregularity. Voxel-based methods [8–10] usually voxelize point clouds into 3D meshes that are then fed into a neural network, which commonly

results in lower-level resolution and high computational cost. However, multi-view based approaches [11–19] capture images of several views of the 3D shape by setting a series of viewpoints, thereby avoiding data hindrance and simplifying 3D shapes into multi-view images. Furthermore, multi-view classification can take 2D image classification networks that perform well, and related methods [17,18] have consequently achieved excellent performance.

Given the multiple views of a 3D shape, several existing methods directly regard the multiple views as sequential views, such as Ma et al. [20], 3D2SeqViews [21] and SeqViews2SeqLabels [16], which considerably rely on complete and regular view configurations, making it difficult to apply them in real practice. A recent multi-views processing method View-GCN [18] constructs a view-graph with multiple views as graph nodes, which has significantly improved the performance. However, both the two above-mentioned strategies utilize all views for network training, resulting in high computational cost; furthermore, redundant views usually lead to redundant features, which directly hinder the performance of multi-view based approaches. For the feature fusing, which is the core focus of the multi-view based approaches, max-pooling is a main pattern which has been employed in several existing methods, such as MVCNN [11], MVCNN-New [13], GVCNN [14], RCPNN [22], View-GCN [18]. However, max-pooling

* Corresponding author.

E-mail address: zhangyuhe0601@nwu.edu.cn (Y. Zhang).

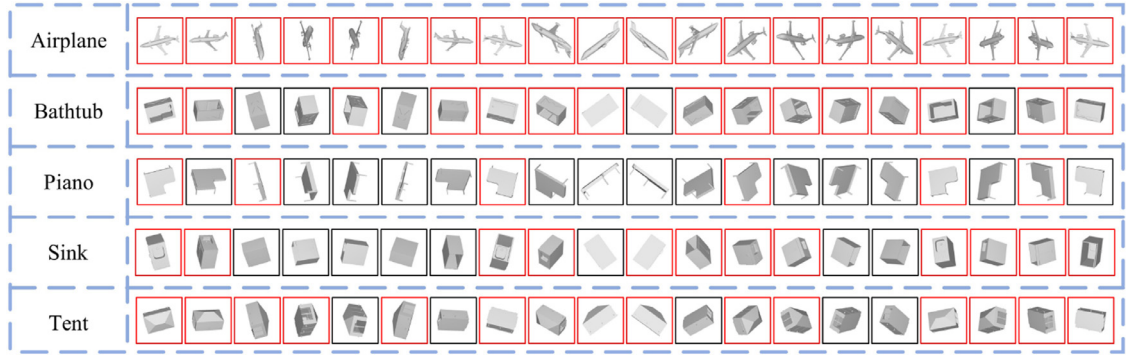


Fig. 1. The views marked in red rectangles are useful, whereas the views marked in black rectangles are redundant, which provide no discriminative information. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

usually leads to feature information loss since the max-pooling operation only preserves the information about the view corresponding to the maximum value and drops other information of the views.

Observation of human vision indicates that an object can be quickly and effectively recognized using a few representative images, and the remaining views of the object are redundant since they cannot provide discriminative information for recognition. As shown in Fig. 1, the airplane can be recognized using all the presented views, whereas the bathtub, piano, sink, and tent can be recognized only using some of the given views. Inspired by this observation, this paper proposes the view-filtering-based multi-view aggregating convolution (VFMVAC) framework to learn multi-view shape descriptors for high-precision 3D model recognition and retrieval. The framework contains two main components. First, a voting-based view filtering algorithm is proposed for the selection of the representative multi-view images. By voting on the multi-view images of a 3D shape, the top-k views are selected to describe the shape, and are then regarded as the input of the rest of the network. The view filtering strategy does not rely on a normalized or complete view configuration when selecting the representative views that are required to recognize shapes, and therefore can significantly improve the memory usage efficiency and reduce the computational cost. Next, a multi-view aggregating module is introduced to integrate the k-view features. This module reshapes the features of the k representative views by introducing a cross-view channel shuffle module. It should be noticed that instead of directly dealing with features from all the views which is employed by previous methods, only the features from the selected k representative views are fed into the cross-view channel shuffle module. This operation considers the combination of the features across the k representative views, thereby allowing for their sufficient fusion. Subsequently, the reshaped feature is fed into the aggregating convolution module for feature fusion, which forms a global shape descriptor that can be used for 3D shape recognition and retrieval. The aggregating convolution operation effectively fuses the k-view features by employing k-channel 2D convolution, which overcomes the information loss of traditional pooling methods during the fusion of multi-view features.

In summary, the main contributions of this research are listed as follows:

- we propose a high-precision 3D shape multi-view recognition framework which can highly promote the performance of the 3D shape classification and retrieval;
- a voting-based view filtering algorithm is proposed; this algorithm can select the most representative views among the existing views to represent 3D shapes, thereby significantly im-

proving memory usage efficiency and reducing the computational cost;

- a novel multi-view aggregating module is designed; in particular, the k-view features are shuffled using a cross-view channel shuffle module that considers the combination of features across views, thereby allowing for their sufficient fusion; furthermore, this module fuses the multi-view features via an aggregating convolution and considers all features of each view, thereby avoiding information loss induced by the traditional pooling methods;
- the proposed framework achieves state-of-the-art recognition and retrieval performance on benchmark datasets.

2. Related works

Shape descriptors are obtained by processing a whole shape, which is different from the surface patch descriptors (such as RoPS [23] and KDD [24]), which only describe the surface patches around key points. Owing to focusing on the whole shape, the shape descriptors are more effective for object recognition, and have developed rapidly in the last decade. Many highly innovative approaches have been proposed where deep learning-based methods [3,8,11] have dominated shape representation. Current deep learning-based methods can be classified into multi-view based, voxel-based, and point-based methods. In this paper, the focus is placed on multi-view based shape representation methods.

Multi-view based methods. Determining how to aggregate multi-view images into a distinguishable global descriptor is the critical challenge for multi-view tasks. As a pioneering work, MVCNN [11] was the first to collect 2D image rendering views of 3D shapes to identify 3D shapes. It uses view-pooling to aggregate features from multi-view images into a single and compact shape descriptor. In later research, more efficient feature fusion methods were gradually explored, and the relationship between views was studied. RPCNN [22] aggregates the features of similar views and then improves the performance of multi-view 3D shape recognition by using recurrent clustering and a pooling layer. MHBN [25] exploits the intrinsic relationship between polynomial kernels and bilinear pooling to obtain a compact global descriptor. Feng et al. [14] proposed a GVCNN in which the views are first grouped, then group-level descriptors are generated, and further aggregated into shape descriptor using group-level descriptors.

Another interesting strategy is to consider multi-view images as a sequence. For example, 3D2SeqViews [21] uses a recursive approach to weight the view content information, and aggregates features via an attention mechanism. Ma et al. [20] used a CNN to extract features for each view, and then used long short-term memory (LSTM) and sequential voting to pool these features. Moreover, VNN [26] and SeqViews2SeqLabels [16] are two other 3D

shape classification methods that consider multi-view images as sequences.

Some particular strategies have achieved good performance. For example, RotationNet [17] uses viewpoint labels as latent variables to estimate the categories and viewpoints of a single viewpoint in an unsupervised way, and aggregates object category predictions for each viewpoint image set. View-GCN [18] proposed by Wei et al., coarsening the graph by local graph convolution, non-local message passing, and selective view sampling.

Multi-view based approaches relied on the fast development of 2D image classification, and now are characterized by optimal 3D shape classification and retrieval performance.

Voxel-based methods. Voxel-based methods usually first involve the transformation of the point cloud into a 3D mesh, followed by the learning of 3D shape features from the 3D mesh to achieve shape classification. Maturana and Scherer [8] introduced a volumetric occupation network called VoxNet to identify 3D objects. Moreover, 3D ShapeNets [9] is a convolutional deep belief network that learns global features from 3D shapes represented by probability distributions of binary variables on a voxel grid. Qi et al. [10] analyzed the performance gap between volumetric CNNs and multi-view CNNs by using a new voxel representation to learn global features and max-pooling to aggregate information from different directions. However, voxel-based methods usually require significant computational overhead, and their shape recognition performance is not as good as that of multi-view based methods.

Point-based methods. In point-based methods, each point of a point cloud is used for feature learning. Point-based methods include point-by-point multilayer perceptron (MLP) methods, convolution-based methods, and graph-based methods. As a pioneering network, PointNet [3] directly uses point clouds as input and aggregates the point features into a global feature using max-pooling. Qi et al. [4] further proposed Pointnet++ to improve the recognition performance by learning local features of point clouds. Liu et al. [5] proposed RSCNN, which learns a high-level relational representation between a sampled point of point set to further the learning of the shape of the point cloud. Liu et al. [27] directly extract 3D keypoint features from point clouds, while preserving the object geometric information as well.

Although point-based methods use point clouds with complete information as input, the further development of point-based methods is seriously affected by factors such as the unstructured and high-dimensional nature of point clouds.

3. Methodology

This section details the architecture of VFMVAC, which is illustrated in Fig. 2. In Section 3.1, the voting strategy for selecting the representative views (top-k views) of a 3D shape is introduced. The voting results are sorted as a new view sequence, and the top-k views are considered as the input of the next phase. Section 3.2 presents the multi-view aggregating module, which includes the cross-view channel shuffle module and the multi-view aggregating convolution module. Additionally, the rest of this section introduces the multi-view configuration and network training details.

3.1. Voting-based view filtering

This subsection introduces the voting module in VFMVAC, and the overall flow is shown in Fig. 3. Given a 3D shape S , N view images are generated from the 3D shape according to the three predefined camera arrays shown in Fig. 4. These view images form a sequence of multi-view images in camera order, which is defined as $\{I_n\}_{n=1}^N$. Each view image I_n in the view sequence is fed

into the CNN to obtain a predicted label l_n for each view, where $l_n \in \{1, 2, \dots, C\}$ and C means C categories in the classification task. Next, the prediction labels $\{l_n\}_{n=1}^N$ are compared with the ground truth labels y :

$$p_n = \begin{cases} 1, & \text{if } l_n = y \\ 0, & \text{if } l_n \neq y \end{cases}, n = 1, 2, \dots, N, \quad (1)$$

where $\{p_n\}_{n=1}^N$ is prediction result. $p_n = 0$ indicates an incorrect prediction, whereas $p_n = 1$ indicates a correct prediction.

Next, two mechanisms for voting are employed. The first is the instance-level voting, in which the prediction results for all shapes as $P = \{p_n^m\}_{n=1, m=1}^{N, M}$, and M is the number of all shapes in the classification task. Instance-level voting is then conducted on the prediction results of all views based on the prediction result P :

$$v_{ins}^n = \sum_{m=1}^M p_n^m. \quad (2)$$

Thus, the predictions of all views with the same viewpoint are cumulative. Next, an auxiliary sequence $A_{ins} = \{n\}_{n=1}^N$ is constructed and then sorts the instance-level voting results $V_{ins} = \{v_{ins}^n\}_{n=1}^N$:

$$V'_{ins} = \text{sort}_{\Delta}((v_{ins}^1, 1), (v_{ins}^2, 2), \dots, (v_{ins}^N, N)), \quad (3)$$

where sort_{Δ} is the sorting function that sorts combination (v_{ins}^n, n) by v_{ins}^n from largest to smallest. The sequence A_{ins} is used to

record the change of position of V_{ins} during the sorting process, which eventually generates the sequence $V'_{ins} = \{n\}_{n \in \{1, 2, \dots, N\}}$. Each number in V'_{ins} represents a view of a viewpoint, i.e., the series of views of instance-level voting was obtained. The instance-level voting mechanism is illustrated in Fig. 5(a).

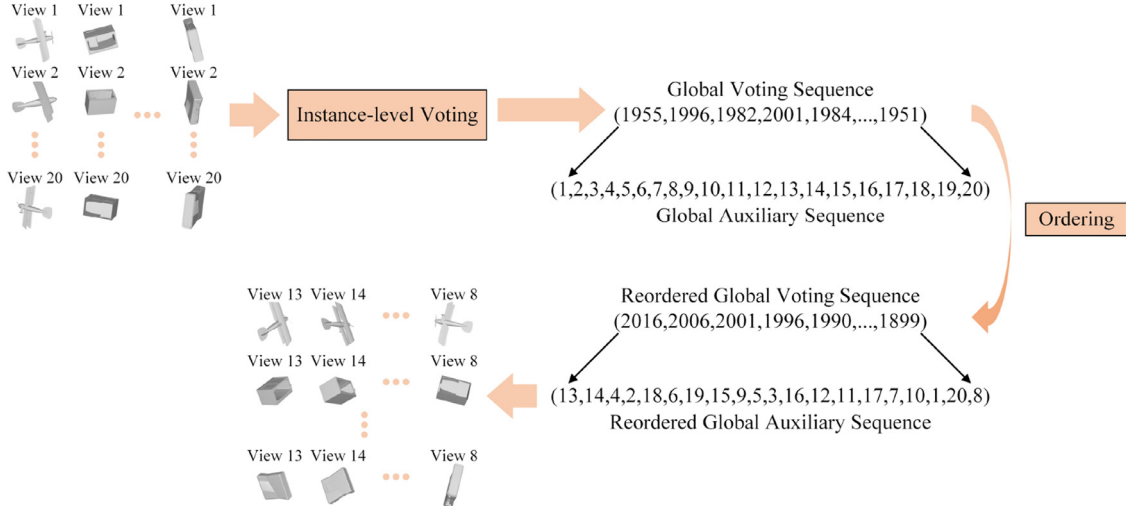
The second voting mechanism is class-level voting, in which the predicted results are denoted as $P = \{p_n^m\}_{n=1, m=1}^{N, M}$ according to the setting in instance-level voting. The prediction labels for all 3D shape views are defined as $L = \{l_n^m\}_{n=1, m=1}^{N, M}$ and the ground truth labels are defined as $Y = \{y^m\}_{m=1}^M$. Class-level voting makes the predictions cumulative for all models in the same category (with the same ground-truth label). However, the class-level voting results are defined as $V_{class} = \{V_{class}^j\}_{j=1}^C$, where $V_{class}^j = \{v_{class}^{j, n}\}_{n=1}^N$ and j means j th category. $v_{class}^{j, n}$ is defined as

$$v_{class}^{j, n} = \sum_{m=1}^M p_n^m \text{ s.t. } y^m = j, \quad (4)$$

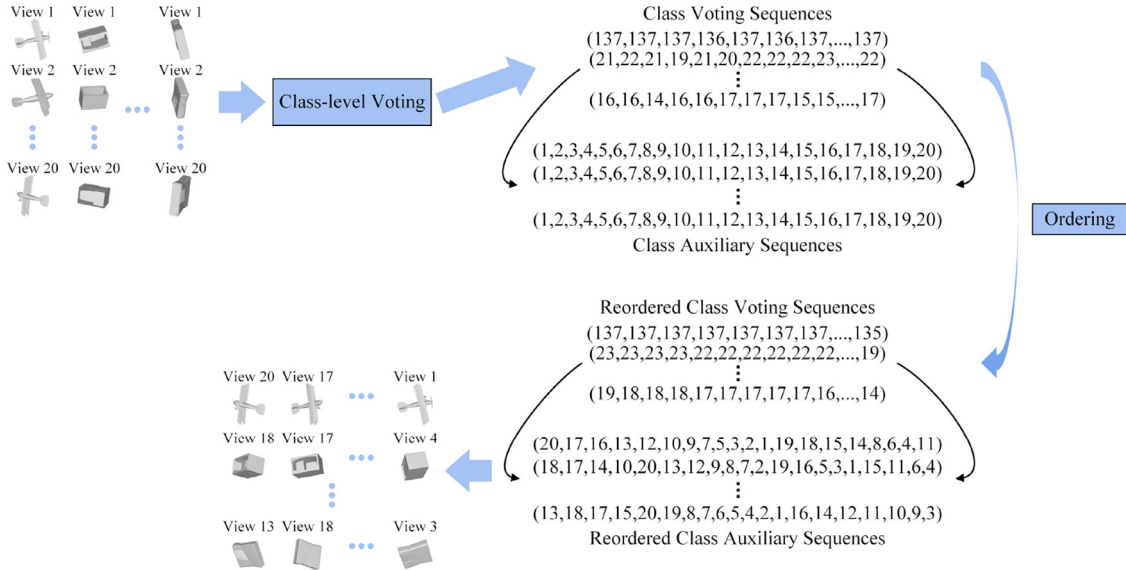
where $y^m = j$ a constraint which makes the predictions cumulative for the same category. Then, the auxiliary sequence $A_{class} = \{A_{class}^j\}_{j=1}^C$ is constructed, where $A_{class}^j = \{n^j\}_{n=1}^N$. The assisting sequence and voting results are then input into the sorting function

$$V'_{class} = \text{sort}_{\Delta}((v_{class}^{j, 1}, 1), (v_{class}^{j, 2}, 2), \dots, (v_{class}^{j, N}, N)), j = 1, 2, \dots, C, \quad (5)$$

after which the sequence set $V'_{class} = \{V'_{class}^j\}_{j=1}^C$ is obtained where $V'_{class}^j = \{n^j\}_{n^j \in \{1, 2, \dots, N\}}$ indicates the view sequence of the category j . Each A_{class}^j in A_{class} corresponds to record the change in position of each V_{class}^j in V_{class} during the sorting process. The class-level voting mechanism is illustrated in Fig. 5(b). Finally, the top-k views in the view sequence are used as the input for the next module.



(a) Instance-level voting has only one voting sequence and one auxiliary sequence.



(b) Class-level voting has voting sequences and auxiliary sequences for each class.

Fig. 5. The two voting mechanisms. The sub-figures respectively correspond to instance-level voting and class-level voting.

determined by the feature extraction network. f_n indicates $f_n = [f_{n1}, f_{n2}, \dots, f_{nc}]$, where $f_{nc} \in \mathbb{R}^{f_h \times f_w}$ and $[\cdot, \cdot]$ is the concatenation operation.

To allow for the information flowing across feature channels and further sufficiently confuse features from different views, the cross-view channel shuffle operation is also introduced, which is illustrated in Fig. 6. Specifically, the cross-view channel shuffle operation first reshapes the k -view features:

$$F^{(0)} = [f_1, f_2, \dots, f_k]. \quad (6)$$

where $F^{(0)} \in \mathbb{R}^{k \times c \times (f_h \times f_w)}$. Then the reshaped features $F^{(0)}$ are transposed, and thereby forming $F^{(1)}$. $F^{(1)}$ is defined as:

$$F^{(1)} = \text{transpose}(F^{(0)}; (2, 1)) = [f'_1, f'_2, \dots, f'_c] \quad (7)$$

where $f'_c \in \mathbb{R}^{k \times (f_h \times f_w)}$, $f'_c = [f_{1c}, f_{2c}, \dots, f_{kc}]$, $\text{transpose}(\dots)$ represents the transpose operation and $(2, 1)$ indicates that $F^{(0)}$ is transposed in the first dimension and the second dimension. Finally, $F^{(1)}$ is also reshaped and the $F' = \{f'_i\}_{i=1}^c$, which is the final output of cross-view channel shuffle, is obtained. The cross-view channel shuffle is implemented by reshape and transpose.

It should be noticed that no parameters in cross-view channel shuffle needs training; moreover, the cross-view channel shuffle is also differentiable, which means it can be embedded into network framework for end-to-end training.

3.2.2. Multi-view aggregating convolution

By performing the cross-view channel shuffle operation, features in $F = \{f_n\}_{n=1}^k$ are reshaped into high-level multi-view feature $F' = \{f'_i\}_{i=1}^c$. An aggregating convolution module is designed to fuse the reshaped feature and then generate a global descriptor of the 3D shape. The aggregating convolution is defined as

$$\hat{F} = \left[\text{convolution}(f'_i; \text{input}, \text{output}, \text{kernel}, \text{padding}) \right]_{i \in \{1, 2, \dots, c\}}, \quad (8)$$

where convolution indicates convolution operation, f'_i is the input to the convolution, input , output , kernel and padding are input channel size, output channel size, convolution kernel size, and padding size, respectively. The convolution kernel size $\text{kernel} = (k_h \times k_w)$ can be adjusted according to the size of the single-view

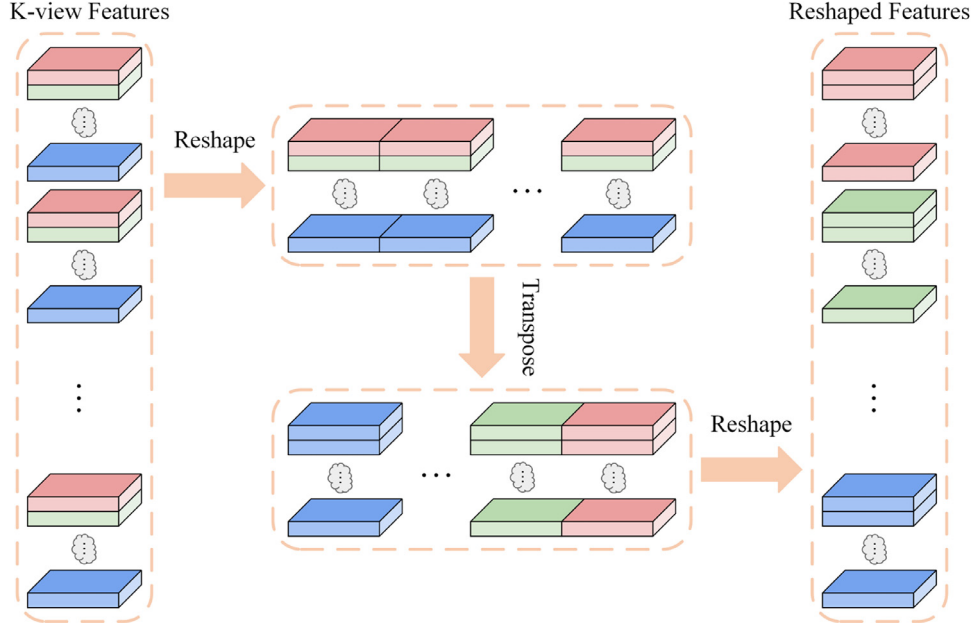


Fig. 6. The features of k-view have k groups and a total of $k \times c$ channels. It is first reshaped to (k, c) , then transposed to (c, k) , and finally reshaped to c groups of features.

feature. The strategy adopted to choose the size of the convolution kernel of the aggregating convolution module, the strategy we adopt is $\lfloor f_h/2 \rfloor \leq k_h \leq \lfloor f_h/2 \rfloor + 1$ and $\lfloor f_w/2 \rfloor \leq k_w \leq \lfloor f_w/2 \rfloor + 1$, and both k_h and k_w are odd numbers. In the implementation, aggregating convolution is designed as a single-layer 2D convolution with input channel of k and an output channel of 1; the padding size is $(\lfloor k_h/2 \rfloor, \lfloor k_w/2 \rfloor)$. Through aggregating convolution, each f'_i in F' is aggregated into a $\hat{f}_i$ and further into $\hat{F} = [\hat{f}_1, \hat{f}_2, \dots, \hat{f}_c]$, where $\hat{F} \in \mathbb{R}^{c \times f_h \times f_w}$.

As shown in Fig. 2, the fused feature $\hat{F}$ is flattened into the one-dimensional vector F_{global} , i.e., the global shape descriptor is obtained. F_{global} is delivered to the fully connected layer to predict the scores of the class labels.

3.3. VFMVAC multi-view configuration

Training loss for VFMVAC. To train VFMVAC for 3D shape classification, the cross-entropy loss function is employed. Specifically, for example, given a global shape descriptor F_{global}^m which is fed into a classifier $\mathcal{G}$, e.g., a fully connected layer with weights $W_{\mathcal{G}} \in \mathbb{R}^{(c \times f_h \times f_w) \times C}$ and followed by a softmax layer, then the VFMVAC total loss for training is:

$$\mathcal{L} = \frac{1}{M} \sum_{m=1}^M \mathcal{L}_m = -\frac{1}{M} \sum_{m=1}^M \sum_{j=1}^C \mathcal{Y}_m \odot \log(\mathcal{G}(F_{global}^m)), \quad (9)$$

where $\mathcal{Y}_m$ is the vector of ground truth labels for F_{global}^m , $\log(\cdot)$ gets the logarithm of each element in the vector, and $\odot$ is the Hadamard product of the two vectors.

Configuration with 12 views. As shown in Fig. 4(a), the 12 virtual cameras are all placed on a circular trajectory and raised by 30 degrees in the vertical direction so that one object has 12 views ($N = 12$). When all 12 views are used as inputs, the input channel k of the aggregating convolution is equal to 12. In this paper, k in top-k views is set as 2, 4, 6, 8, 10.

Configuration with 20 views. Figure 4(b) shows that the virtual camera is placed on the vertex of a dodecahedron ($N = 20$) containing the object. For the 20-view version, all views of a 3D shape are input when $k = N$. For the top-k views, k is set to 2, 4, ..., 18.

Configurations for different backbone networks. Three different backbone networks are used for the implementation of VFMVAC, namely AlexNet [30], ResNet-18 [28], and ResNet-50 [28]. The 2D view image size for all backbone network input is $I_n \in \mathbb{R}^{3 \times 224 \times 224}$. For the single-view feature $f_{AlexNet} \in \mathbb{R}^{256 \times 6 \times 6}$ extracted by AlexNet. Following the convolution kernel selection strategy outlined in Section 3.2.2, the size is $K = 3$. Moreover, for ResNet-18 and ResNet-50, the features extracted for the individual views are $f_{ResNet-18} \in \mathbb{R}^{512 \times 1 \times 1}$ and $f_{ResNet-50} \in \mathbb{R}^{2048 \times 1 \times 1}$, respectively. The convolution kernel size is set to $K = 1$.

3.4. Network training

Training method. The training of the proposed framework was divided into three steps. In the first step, the “training set” (for data sets without validation sets, the original training set is divided into “training set” and validation set) was used to fine-tune the ImageNet [29] pre-trained ResNet-18, and the view sequences for both voting methods in the validation set were then obtained. The last two training steps of VFMVAC are similar to that of MVCNN-New [13]. In the second step, ResNet-18 on ImageNet was fine-tuned on shuffled multi-view 2D images of all training 3D shapes for view feature extractor. The third training step was divided into two cases; the base version was trained and tested with 20 views of each shape, while the voting version was trained with a random input of k views in the training stage, and the top- k views of both view sequences were used as the input in the testing stage. The entire architecture, including the backbone network feature extractor and VFMVAC, was trained via an end-to-end approach.

Training details. For the first two training steps of the feature extraction network, the stochastic gradient descent (SGD) optimizer was used, the momentum, weight decay, batch size, and initial learning rate of which were respectively set as 0.9, 0.001, 200, and 0.01. In total, 30 epochs were trained, and the learning rate decayed by half every 10 epochs. During the training of the entire network architecture, the SGD optimizer with momentum was used, but the batch size, initial learning rate, and the number of epochs were changed to 10, 0.001, and 15, respectively. Each batch contains 10 shapes of multi-view images for the basic training, with 200 and 120 views for the 20-view and 12-view

Table 1

Training time for each step (m and h represent minutes and hours, respectively).

Step	ModelNet40	ShapeNet Core55	RGBD
1st	1.7h	7.7h	2.5m
2nd	2.2h	7.7h	3.2m
3rd	1.1h	4.0h	1.6m

versions, respectively. For the top- k version of the training, each batch contains $10 \times k$ views. Inspired by reference [31], a learning rate warm-up strategy was also employed. Specifically, the learning rate increased linearly from 0 to 0.002 in the first epoch (originally it was increased to 0.001), allowing for the acceleration of the network convergence during the training process. Then the learning rate decayed gradually from 0.001 to 0, where the decaying ratio of each epoch is computed by a cosine function. Our approach was implemented by PyTorch¹ and used mixed precision training [32] in all stages of training.

4. Experiments

Shape classification and retrieval experiments were conducted on the ModelNet40 [9], ShapeNet Core55 [33], as well as a real image benchmark dataset [34]. To evaluate the performance of our method, an NVIDIA GTX 1080 Ti GPU, Intel Core i7-7820X CPU, and 64GB RAM were used.

4.1. Datasets

ModelNet40. The ModelNet40 dataset contains 12,311 3D shapes, of which 9843 were used for training and 2468 were used for testing. ModelNet40 is divided into 40 categories, each containing a different number of 3D shapes.

ShapeNet Core55. ShapeNet Core55 contains 51,162 3D models divided into 55 categories, which can be further subdivided into 203 subcategories. The training set, validation set, and test set include 35,764, 5133, and 10,265 shapes, respectively. The performance of the proposed method was evaluated by the “normal” datasets (in which all models are normalized to the unit length of the square).

RGBD. RGBD is a real captured multi-view dataset consisting of 300 household objects grouped into 51 categories. The image sequences consist of RGB and RGBD images. In this study, the proposed method was evaluated using the RGB images.

The training times per step of VFMVAC on all views of ModelNet40, ShapeNet Core55, and RGBD datasets are listed in Table 1. It should be noted that the training time was affected by hardware configuration and other factors (such as the dataset size and training configuration).

4.2. 3D shape classification on ModelNet40

4.2.1. The effectiveness of VFMVAC

The performance of VFMVAC was first investigated with three different backbone networks, namely AlexNet [30], ResNet-18 [28], and ResNet-50 [28], using the top- k mechanism. The classification results on the ModelNet40 dataset under various voting frameworks are reported in Table 2, and Fig. 7 presents the reordered multi-view images of some models obtained by the voting algorithm when using ResNet-18 as backbone network. “Random” represents the classification accuracies of randomly selecting k views from the initial sequence of 20 views, and then selecting $k/2 + 1$

views from the class-level voting sequence of k views. The evaluation metric is the per-instance accuracy, and the best scores for each framework under different voting algorithms are indicated in bold.

With instance-level voting, all three backbone networks achieved the highest accuracy at top-18. Both the ResNet-18 and AlexNet backbone networks achieved the highest accuracy at different k values with the Random and class-level voting. In terms of the averaged values, ResNet-50 was the best with comparable performance to that of ResNet-18. Different values of k were found to have a dramatic impact on the performance of 3D model classification, which indicates that view filtering is a critical factor in multi-view based 3D object classification.

It also can be seen that class-level voting was superior to instance-level voting, which was also superior to the Random. Under the class-level voting mechanism with higher reliability, the 3D model classification performance of VFMVAC increased and then decreased with the decrease of the k -value for different backbone networks. AlexNet, ResNet-18, and ResNet-50 achieved the best performance when $k = 16, 10, 8$, respectively. These observations indicate that the proposed voting algorithm can filter out the redundant views from the multi-view. Still, when the value of k was less than a certain threshold, the performance of VFMVAC began to degrade, indicating that the representative views in the multi-view were discarded. Nevertheless, the Random results illustrate that the proposed method still achieved good performance for random missing views, and achieved a maximum per-instance classification accuracy of 97.45%.

4.2.2. The performance of classification

This section compares various 3D shape classification methods, and primarily focuses on the multi-view based methods. The detailed data are listed in Table 3. Among the multi-view based methods, MVCNN [11], MVCNN-new [13], GVCNN [14] all fuse multi-view features by max-pooling. Compared with these other methods, VFMVAC achieves significantly higher accuracies by more than 4.40% per class and 2.97% per instance accuracies. Furthermore, compared with the methods that utilize the relationships of sequential views, such as 3D2SeqViews [21], SeqViews2SeqLabels [16], Ma, et al. [20], VFMVAC can be adapted to more flexible view configurations (as shown in Section 4.5) and exhibited accuracy improvements of 5.29% per class and 4.57% per instance. Compared with RotationNet [17] and EMV [35], which explore more general view configurations, VFMVAC exhibited accuracy improvements of 0.51% per class and 0.60% per instance. Among these existing approaches, View-GCN [18] achieved the state-of-the-art performance, which filters the multiple views by a hierarchy of view graphs and fuses multi-view features via the max-pooling. However, VFMVAC still outperformed View-GCN by 0.30% and 0.37% in terms of the per-class and per-instance accuracies, owing to the application of voting-based view filtering and the multi-view aggregating module.

To further validate the superiority of VFMVAC, it was also compared with point-based and voxel-based, including that proposed by PointNet [3], PointNet++ [4], RS-CNN [5], DenX-Conv [6], VoxNet [8], 3DShapeNets [9], MVCNN-MultiRes [10]. The experimental results listed in Table 3 also demonstrate that VFMVAC exhibited respective improvements of 5.40% and 4.17% in terms of the per-class and per-instance accuracies.

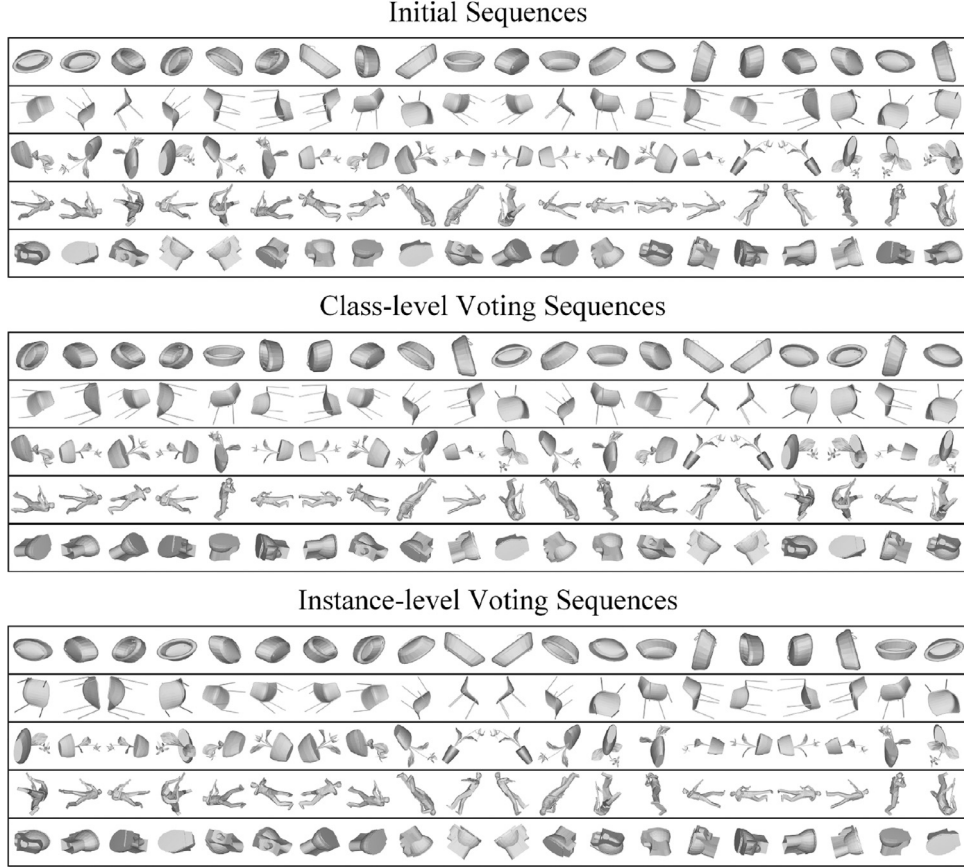
To validate the generalizability of our VFMVAC, the performance of VFMVAC was also compared with those of RotationNet [17] and View-GCN [18], using AlexNet and ResNet-50 as backbone networks. The detailed data are listed in Table 4. With AlexNet as backbone network, VFMVAC achieved improvements 0.85% and 0.04% in terms of the per-instance accuracy as compared to RotationNet and View-GCN, respectively, and when using ResNet-50 as

¹ <https://pytorch.org/>

Table 2

Comparison of classification accuracies (%) with various k values and different backbone networks.

Archit.	Voting	Top-k									Mean±Std.
		2	4	6	8	10	12	14	16	18	
AlexNet	Random	91.86	96.11	96.03	96.19	96.00	95.91	96.11	96.52	96.35	95.68±1.36
	Instance	93.80	95.95	96.27	96.64	96.68	96.56	96.60	96.80	97.00	96.26±0.91
	Class	94.98	96.23	96.64	96.76	97.00	96.96	97.08	97.16	97.00	96.65±0.65
ResNet-18	Random	94.43	97.08	97.45	97.24	96.07	96.68	97.24	97.16	96.84	96.69±0.89
	Instance	96.35	97.08	97.20	97.33	97.41	97.41	97.57	97.41	97.61	97.26±0.36
	Class	96.56	97.41	97.37	97.65	97.97	97.69	97.57	97.57	97.65	97.49±0.37
ResNet-50	Random	94.81	97.16	97.24	97.32	95.83	95.58	96.88	96.80	95.58	96.36±0.86
	Instance	96.68	97.24	97.29	97.41	97.41	97.33	97.37	97.57	97.65	97.33±0.26
	Class	97.08	97.37	97.77	97.93	97.61	97.73	97.77	97.61	97.53	97.60±0.24

**Fig. 7.** Initial views sequences of ModelNet40 partial models, and reordered views sequences using two voting algorithms.

backbone network, the per-instance accuracy presented respective improvements of 1.01% and 0.63%.

4.2.3. Parameters and memory usage

This subsection evaluates the number of parameters and memory usage of different methods, and the results are listed in Table 5. Although the number of parameters of VFMVAC is greater than those of VoxNet and PointNet, VFMVAC (AlexNet) exhibited the most efficient memory usage and the accuracy dramatically higher than those of VoxNet and PointNet. VFMVAC (ResNet-18) has only about one-third of the number of parameters of ViewGCN (ResNet-18), it still exhibited better performance. The data reported in Table 5 reveal that VFMVAC exhibits advantages in memory usage, parameters, and performance as compared to other approaches [13,17].

4.3. ShapeNet Core55

4.3.1. The performance of classification

ShapeNet Core55 includes more shapes and varieties than ModelNet40, and the data for MVCNN, and the imbalance between categories in the training set further increases the classification difficulty. Therefore, the classification performance of VFMVAC was also tested on ShapeNet Core55 [33]. Because ShapeNet Core55 is usually used for 3D retrieval experiments, and the results of classification experiments on this dataset have rarely been made public, the proposed method was not compared with other methods.

VFMVAC (ResNet-18) was trained and tested with the 20-view configuration of all training and test sets. VFMVAC achieved a classification accuracy of 89.34% per instance, which validates its good classification performance on ShapeNet Core55.

Table 3
Comparison of classification accuracy (%).

ModelNet40			
Method	Input	Per Class	Per Instance
PointNet [3]	Points	86.2	89.2
PointNet+ [4]		-	91.9
RS-CNN [5]		-	93.6
DenX-Conv [6]		-	92.5
VoxNet [8]	Voxels	83.0	-
3D ShapeNets [9]		77.32	-
MVCNN-MultiRes [10]		91.4	93.8
MVCNN [11]	Images	90.1	-
MVCNN-new [13]		92.4	95.0
GVCNN [14]		90.7	93.1
3D2SeqViews [21]		91.51	93.40
SeqViews2SeqLabels [16]		91.12	93.31
Ma et al. [20]		-	91.05
RotationNet (VGG-M) [17]		96.29	97.37
EMV [35]		-	94.67
View-GCN (ResNet-18) [18]		96.5	97.6
Ours (ResNet-18)		96.67	97.93
Ours (ResNet-18+Ins. Top-k)		96.54	97.61
Ours (ResNet-18+Class Top-k)		96.80	97.97

Table 4
Comparison with RotationNet [17] and View-GCN [18], using AlexNet and ResNet-50 as backbone networks.

Method	Backbone	Per Instance (%)
RotationNet	AlexNet	96.39
View-GCN		97.2
Ours		97.24
Ours (Ins. Top-k)		97.00
Ours (Class Top-k)		97.16
RotationNet	ResNet-50	96.92
View-GCN		97.3
Ours		97.69
Ours (Ins. Top-k)		97.65
Ours (Class Top-k)		97.93

Table 5
Comparison of the number of parameters, memory usage (training batch size is 64, which means that the input for training is 64 images), and classification accuracy (number in brackets show per class accuracy) on ModelNet40.

Algorithm	# Params (M)	Memory (GB)	Accuracy (%)
MVCNN-New (VGG-11) [13]	128.9	10.0	95.0(92.4)
PointNet [3]	3.5	4.4	89.2(86.1)
VoxNet [8]	1.4	2.0	85.6(81.4)
RotationNet (VGG-M) [17]	102.2	5.3	97.4(96.3)
View-GCN (ResNet-18) [18]	33.9	-	97.6(96.5)
Ours (AlexNet)	48.70	1.72	97.24(95.81)
Ours (ResNet-18)	11.72	1.85	97.93 (96.67)
Ours (ResNet-50)	24.84	4.26	97.69(96.91)

¹ The training input of 64 images is implemented in two ways (top-8 × 8, top-16 × 4) with consistent memory usage.

² Data for MVCNN-New, PointNet, and VoxNet are derived from Su et al. [13].

4.3.2. The performance of retrieval

The retrieval performance of the proposed method was tested on ShapeNet Core55. For the test, shape retrieval was respectively conducted with 20 views and the top-k views determined by class-level voting as the input, and ResNet-18 was used as the backbone network. For a query, all objects with the same predicted class label were firstly searched, and the probability scores for class label prediction were sorted. The retrieval results were then re-ranked so that samples with the same query subcategory labels were ranked higher than the others. Following the suggestion of

reference [33], the top 1000 retrieval results for each query were returned.

For a fair comparison, officially published evaluation codes² were utilized. The evaluation criteria are divided into two versions, namely the micro-averaged and macro-averaged versions. The micro-averaged version treats each retrieved result equally across categories, while the macro-averaged version provides an unweighted average of the entire dataset. The evaluation metrics for this dataset included the precision (P@N), recall (R@N), F-score (F1@N), mean average precision (mAP), and normalized discounted cumulative gain (NDCG). For more details, please refer to the work by reference [33].

The proposed method was compared with various methods that participated in SHREC2017, the top two methods that participated in SHREC2016 and View-GCN [18]. As shown in Table 6, VFMVAC outperformed RotationNet [17], the top-ranked method in SHREC2017, on all metrics except the micro-averaged NDCG. Furthermore, VFMVAC achieved significantly higher accuracy than other multi-view methods, such as Improved-GIFT [36], ReVGG, MVFusionNet, CM-VGG5-6DB and GIFT [37], and voxel-based methods, such as DLAN, ZFDR and DeepVoxelNet. The MVCNN [11] is a baseline method that uses view-pooling to fuse features. However, VFMVAC aggregates multi-view features by cross-view channel shuffle and aggregating convolution module, and therefore compared with MVCNN, the retrieval performance of VFMVAC in terms of the micro-averaged and macro-averaged mean average precision (mAP) was improved by 4.3% and 4.6%, respectively. Furthermore, View-GCN [18] is a state-of-the-art method, compared to which we have achieved improvements in all metrics except micro-averaged mAP, e.g., with an improvement of 2.9% on macro-averaged R@N. Some retrieval results of the proposed method are presented in Fig. 8.

4.4. Real multi-view images classification

The classification performance of VFMVAC was also evaluated on a benchmark dataset consisting of real captured multi-view images. Using the same setup as described in a previous study [17], for each object, an elevation angle of 45° was selected, and one image per 30° of rotation was chosen for a total of 12 images. To ensure fairness, experiments were first conducted using the officially offered proportions³ to split the training set and test set. Then, one shape per category was randomly chosen as the test set, and the remaining shapes as the training set, as suggested by reference [34]. For both splitting versions, ten-fold cross-validation was performed to report the average results, as suggested by reference [34].

The experimental results are presented in Table 7. Using AlexNet as backbone network, the per-instance accuracy of VFMVAC was 3.53% and 0.30% higher than those of MVCNN [11] and RotationNet [17], respectively. Moreover, VFMVAC (ResNet-50) achieved best accuracy of 90.98% with class-level voting.

Experiments were then conducted based on the random splitting of the dataset; three random seeds were used to divide the training and test sets. The average of the three accuracy values was considered the final evaluation metric, and the results are reported in Table 8. Compared with View-GCN (ResNet-18) [18], VFMVAC (ResNet-18) exhibited improvements of 0.60% in terms of the per-instance classification accuracy. Compared with some previous methods, such as MDSI-CNN [38], CFK [39], and MMD-CNN [40], which require more views, VFMVAC exhibited obvious improvements and achieved the optimal per-instance classification

² <https://shapenet.cs.stanford.edu/shrec17/code/evaluator.zip>

³ http://rgbd-dataset.cs.washington.edu/dataset/rgbd-dataset_eval/testinstance_ids.txt

Table 6
Shape retrieval results (%) on ShapeNet Core55.

Method	microALL					macroALL				
	P@N	R@N	F1@N	mAP	NDCG	P@N	R@N	F1@N	mAP	NDCG
ZFDR	53.5	25.6	28.2	19.9	33.0	21.9	40.9	19.7	25.5	37.7
DeepVoxNet	79.3	21.1	25.3	19.2	27.7	59.8	28.3	25.8	23.2	33.7
DLAN	81.8	68.9	71.2	66.3	76.2	61.8	53.3	50.5	47.7	56.3
RotationNet [17]	81.0	80.1	79.8	77.2	86.5	60.2	63.9	59.0	58.3	65.6
Improved GIFT [36]	78.6	77.3	76.7	72.2	82.7	59.2	65.4	58.1	57.5	65.7
ReVGG	76.5	80.3	77.2	74.9	82.8	51.8	60.1	51.9	49.6	55.9
MVFusionNet	74.3	67.7	69.2	62.2	73.2	52.3	49.4	48.4	41.8	50.2
CM-VGG5-6DB	41.8	71.7	47.9	54.0	65.4	12.2	66.7	16.6	33.9	40.4
GIFT [37]	70.6	69.5	68.9	64.0	76.5	44.4	53.1	45.4	44.7	54.8
MVCNN [11]	77.0	77.0	76.4	73.5	81.5	57.1	62.5	57.5	56.6	64.0
View-GCN [18]	81.8	80.9	80.6	78.4	85.2	62.9	65.2	61.1	60.2	66.5
Ours	82.3	81.5	81.4	77.2	85.6	64.0	68.1	63.7	61.2	68.2
Ours (Class Top-k)	82.2	82.0	81.5	77.8	86.2	63.2	67.6	63.2	60.7	67.7

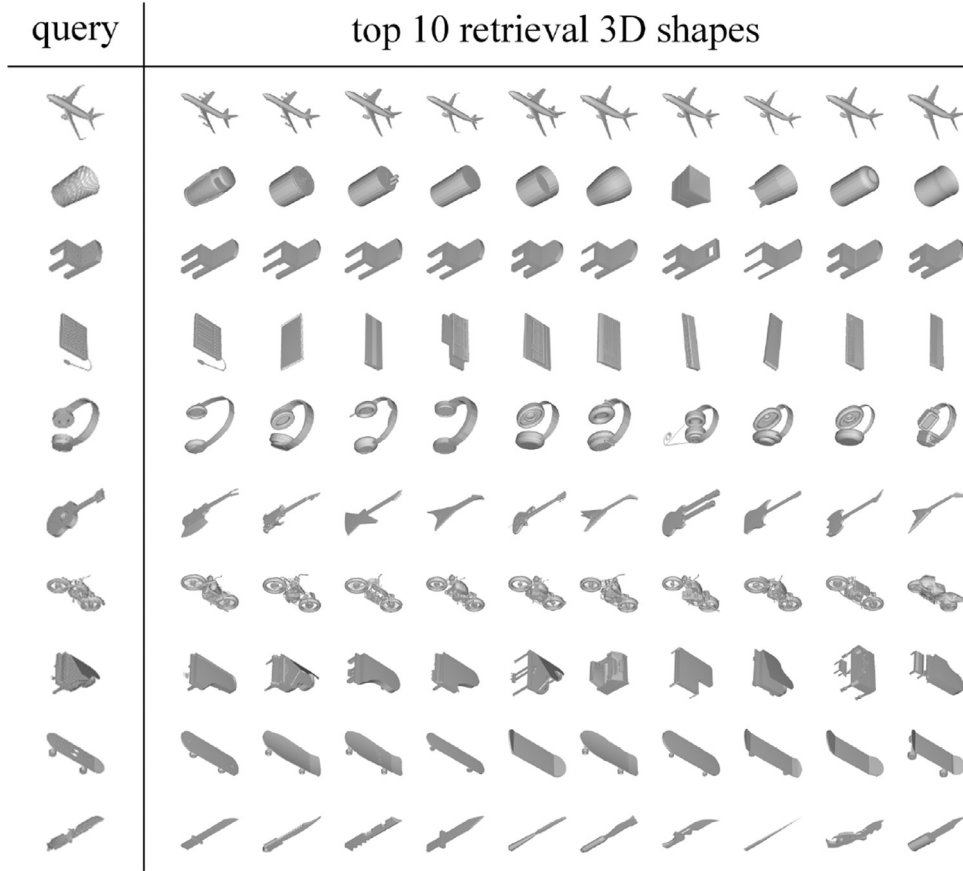


Fig. 8. ShapeNet Core55 portion of the retrieval results.

accuracy results. These findings validate that the proposed VFM-VAC can perform well on real captured multi-view images.

4.5. Effectiveness analysis on VFMVAC

The effectiveness of VFMVAC was evaluated on ModelNet40 [9]. All experiments were conducted with ResNet-18 [28] as backbone network; furthermore, the experimental settings was the same as that described in Section 3.3, and both the 12-view and 20-view configurations were employed. The baseline v1 was the MVCNN-New [13] which uses max-pooling for fusing the multi-view features and the baseline v2 was a variance of MVCNN-New, in which the max-pooling layer was replaced by the proposed multi-view

aggregating module for fusing multi-view features. Furthermore, the comparison was also conducted on View-GCN [18] which has achieved the state-of-the-art performance.

Effectiveness of view filtering algorithm. To reveal the superiority of the voting algorithm, experiments were conducted on baseline v1. The results are listed in Table 9 which demonstrates that the baseline v1 exhibited a slight improvement under both the instance-level voting and class-level voting; furthermore, the class voting also outperforms the instance voting. Moreover, it also can be seen from Table 10 that the performance of the baseline v1 was improved at different k values. These results validate that the proposed voting mechanism can filter out redundant views and effectively improve multi-view recognition performance.

Table 7

The classification accuracy (%) on the training/test split provided by the official.

Method	# View	Per Instance
MVCNN (AlexNet) [11]	12	86.08
RotationNet (AlexNet) [17]	12	89.31
Ours (AlexNet)	12	89.61
Ours (AlexNet+Class Top-k)	6	89.41
Ours (ResNet-18)	12	90.00
Ours (ResNet-18+Class Top-k)	10	90.20
Ours (ResNet-50)	12	90.00
Ours (ResNet-50+Class Top-k)	8	90.98

¹ Data for MVCNN, and RotationNet are derived from Kanazaki et al. [17].

Table 8

The classification accuracy (%) on the training/test random splitting.

Method	# View	Per Instance
MDSI-CNN [38]	≥ 80	89.9
CFK [39]	≥ 80	86.8
MMDCNN [40]	≥ 80	89.5
View-GCN(ResNet-18) [18]	12	94.3
Ours (AlexNet)	12	92.16
Ours (ResNet-18)	12	94.90
Ours (ResNet-50)	12	93.92

Table 9

Comparison of classification accuracies (%) with various k values and different view orders of Baseline v1.

View Order	Top-k					Mean $\pm$ Std.
	2	4	6	8	10	
Original	94.17	95.01	94.98	94.65	95.50	94.86 $\pm$ 0.44
Instance	95.06	95.22	95.18	95.83	95.79	95.42 $\pm$ 0.33
Class	95.34	95.62	95.66	95.99	96.11	95.74$\pm$0.28

Table 10

Comparison of classification accuracy (%) of Baseline v1 on all views and selected top-k views.

# Views	Per Class	Per Instance
20	96.56	97.45
Ins. Top-18	96.31	97.53
Class Top-10	96.53	97.69
Class Top-12	97.01	97.69
Class Top-16	96.46	97.49
Class Top-18	96.54	97.57

Table 11

Results (%) of methods with different pre-training configurations.

Pre-training		Method			
ImageNet	Single View	Baseline v1	Baseline v2	View-GCN [18]	Ours
		94.61	93.72	93.11	94.04
✓		96.47	96.56	96.80	96.88
	✓	96.39	96.68	96.19	97.04
✓	✓	97.45	97.73	97.60	97.93

Effectiveness of multi-view aggregating module. Since the performance of the multi-view classification method is considerably affected by pre-training, the experiments were conducted on the baseline v1 and v2, View-GCN and the proposed VFMVAC under different pre-training configurations. Table 11 summarizes the data, from which the proposed VFMVAC still produced the best performance scores, except for the configuration without any pre-training. Furthermore, it was also found that the baseline

Table 12

Results (%) on perturbed data. The perturbing patterns include clutter curves and dots (CCD), Gaussian noise (GN), random erasing (RE) and random resized crop (RRC).

Perturbing Pattern				Method			
CCD	GN	RE	RRC	Baseline v1	Baseline v2	View-GCN [18]	Ours
✓				97.20	97.45	97.16	97.57
	✓			97.12	97.29	96.96	97.41
		✓		97.24	97.49	97.53	97.85
			✓	97.37	97.53	97.57	97.89
✓	✓			97.16	97.53	97.24	97.61

Table 13

Results (%) on irregular view configuration.

Method	# View	Per Instance
Baseline v1	12	93.68
View-GCN [18]	12	93.40
Baseline v2	12	94.25
Ours	12	94.69
Ours (Ins. Top-k)	8	94.49
Ours (Class Top-k)	8	95.38

v2 with different pre-training configurations exhibited improvements compared with baseline v1, which demonstrate that the proposed multi-view aggregating module can efficiently integrate the features of each view and thus avoid information loss. Moreover, when baseline v2 takes 20 views as input, the baseline v2 has only 21 more parameters than baseline v1, which also demonstrates the efficiency of the proposed multi-view aggregating module.

Robustness to perturbed data. Table 12 presents the results of baseline v1 and v2, View-GCN and the proposed VFMVAC on the perturbed views. The perturbing patterns include clutter curves and dots (CCD), Gaussian noise (GN) with mean $\mu = 0$ and standard deviation $0.01 \leq \sigma \leq 0.10$, random erasing (RE) and random resized crop (RRC). As presented in Table 12, VFMVAC is robust to all the perturbing patterns and still achieved the best performance. Furthermore, compared with baseline v1, VFMVAC exhibited respective improvements of 0.37%, 0.29%, 0.61%, 0.52% and 0.45% in terms of the per-instance accuracies. Moreover, the improvements between baseline v1 and v2 also indicate that the multi-view aggregating module is more robust than max-pooling.

Robustness to irregular view configuration. We conducted the experiments with irregular view configuration which is shown in Fig. 4(c). The numbers of the representative views are 8 and 12, respectively. From Table 13, VFMVAC was found to outperform the other baselines, and with the class-level voting algorithm, the classification performance of VFMVAC was found to be further improved. Our method with the satisfactory performance on the irregular view configuration also highlighted the great potential for real-world applications.

Computational efficiency and complexity analysis. Table 14 summarizes the inference time⁴ and computational (floating-point operations/sample) complexity of VFMVAC. The inference time with 20 views of View-GCN [18] is approximately 1.1 x that of VFMVAC. On average, the voting module significantly reduced inference times by approximately 49% compared to taking all views as input, also the floating-point in the feature extraction part by 50%. Moreover, the VFMVAC dramatically reduced the floating-point in the feature classification part by 99.91% compared to the View-GCN. Moreover, VFMVAC is much more scalable - its inference time and computational complexity are $O(N)$ - linear in the

⁴ Evaluation code: <https://github.com/jcjohnson/cnn-benchmarks>.

Table 14

Inference time and computational complexity of different architectures for multi-view classification. The results of the top-k views are represented by *mean ± std.*. The "ms" and "M" respectively stand for millisecond and million.

Method	Inference Time (ms)	FLOPs/Sample (M)	
		Feature Extraction	Feature Classification
Baseline v1	8.28	36371.09	0.02
View-GCN [18]	9.08	36371.09	621.83
Baseline v2	8.31	36371.09	0.03
Ours	8.31	36371.09	0.56
Ours (Top-k)	4.26±2.13	18185.54±9390.97	0.55±0.00

number of input views. Empirically, VFMVAC (Top-10) can deal with over 0.2k objects per second for classification using a 1080Ti GPU, showing great potential for real-time applications.

5. Conclusion

This paper proposed the view-filtering-based multi-view aggregation convolution (VFMVAC) to overcome the limitations of multi-view classification, e.g., relying on regular view configurations, having a large floating-point and long inference times, etc. VFMVAC first employed a voting-based view filtering algorithm that can filter out representative views from only existing views (with complete or missing view configuration), thereby making multi-view approaches available for more complex scenarios, such as recognizing multiple views captured by randomly generated viewpoints and random missing views. Furthermore, the voting module can effectively reduce inference time, floating-point and memory usage, etc., demonstrating the great potential of VFMVAC for real-time applications. VFMVAC also contains a multi-view aggregating module which consists of cross-view channel shuffle and aggregating convolution. The cross-view channel shuffle module enables information to flow across-view and makes it easy to fuse multi-view features. The aggregating convolution module is implemented by a 2D convolution of multiple channels so that each selected view is taken into consideration, thereby avoiding feature information loss due to the pooling methods. Extensive experiments have shown that each component of VFMVAC has a positive impact on multi-view recognition and that VFMVAC achieves state-of-the-art 3D shape recognition and retrieval performance. Although VFMVAC learns multi-view with good performance, it still suffers from the problem that the training time is long with the first two stages. In future work, an interesting and meaningful direction of research would be the cross-modal feature fusion of point cloud and multi-view.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the [National Natural Science Foundation of China](#) (NSFC) (61802311, 61772421, 61731015, 61802244, 61602380), Key R&D Program of Shaanxi Province (2019SF-272), Natural Science Basic Research Plan in Shaanxi Province of China (2019JQ-668), and the [Fundamental Research Funds for the Central Universities](#) (GK201903114).

Supplementary material

Supplementary material associated with this article can be found, in the online version, at [10.1016/j.patcog.2022.108774](https://doi.org/10.1016/j.patcog.2022.108774).

References

- [1] C.K. Chow, S.Y. Yuen, Illumination direction estimation for augmented reality using a surface input real valued output regression network, *Pattern Recognit.* 43 (4) (2010) 1700–1716, doi:[10.1016/j.patcog.2009.10.008](https://doi.org/10.1016/j.patcog.2009.10.008).
- [2] T. Pylvanainen, K. Roimela, R. Vedantham, J. Itaranta, R. Grzeszczuk, Automatic alignment and multi-view segmentation of street view data using 3D shape priors, in: *Symposium on 3D Data Processing, Visualization and Transmission (3DPVT)*, vol. 737, 2010, pp. 738–739.
- [3] R.Q. Charles, H. Su, M. Kaichun, L.J. Guibas, PointNet: deep learning on point sets for 3D classification and segmentation, in: *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 77–85, doi:[10.1109/CVPR.2017.16](https://doi.org/10.1109/CVPR.2017.16).
- [4] C.R. Qi, L. Yi, H. Su, L.J. Guibas, PointNet++: deep hierarchical feature learning on point sets in a metric space, vol. 30, 2017, pp. 5099–5108.
- [5] Y. Liu, B. Fan, S. Xiang, C. Pan, Relation-shape convolutional neural network for point cloud analysis, in: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 8887–8896, doi:[10.1109/CVPR.2019.00910](https://doi.org/10.1109/CVPR.2019.00910).
- [6] J. Lee, S.-U. Cheon, J. Yang, Connectivity-based convolutional neural network for classifying point clouds, *Pattern Recognit.* 112 (2021) 107708, doi:[10.1016/j.patcog.2020.107708](https://doi.org/10.1016/j.patcog.2020.107708).
- [7] H. Wang, Y. Zhang, W. Liu, X. Gu, X. Jing, Z. Liu, A novel GCN-based point cloud classification model robust to pose variances, *Pattern Recognit.* 121 (2022) 108251, doi:[10.1016/j.patcog.2021.108251](https://doi.org/10.1016/j.patcog.2021.108251).
- [8] D. Maturana, S. Scherer, VoxNet: a 3D convolutional neural network for real-time object recognition, in: *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 922–928, doi:[10.1109/IROS.2015.7353481](https://doi.org/10.1109/IROS.2015.7353481).
- [9] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, J. Xiao, 3D ShapeNets: a deep representation for volumetric shapes, in: *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1912–1920, doi:[10.1109/CVPR.2015.7298801](https://doi.org/10.1109/CVPR.2015.7298801).
- [10] C.R. Qi, H. Su, M. Niener, A. Dai, M. Yan, L.J. Guibas, Volumetric and multi-view CNNs for object classification on 3D data, in: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 5648–5656, doi:[10.1109/CVPR.2016.609](https://doi.org/10.1109/CVPR.2016.609).
- [11] H. Su, S. Maji, E. Kalogerakis, E. Learned-Miller, Multi-view convolutional neural networks for 3D shape recognition, in: *2015 IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 945–953, doi:[10.1109/ICCV.2015.114](https://doi.org/10.1109/ICCV.2015.114).
- [12] A.-A. Liu, W.-Z. Nie, Y. Gao, Y.-T. Su, Multi-modal clique-graph matching for view-based 3D model retrieval, *IEEE Trans. Image Process.* 25 (5) (2016) 2103–2116, doi:[10.1109/TIP.2016.2540802](https://doi.org/10.1109/TIP.2016.2540802).
- [13] J.-C. Su, M. Gadelha, R. Wang, S. Maji, A deeper look at 3D shape classifiers, in: *COMPUTER VISION - ECCV 2018 WORKSHOPS, PT III*, vol. 11131, 2019, pp. 645–661, doi:[10.1007/978-3-030-11015-4_49](https://doi.org/10.1007/978-3-030-11015-4_49).
- [14] Y. Feng, Z. Zhang, X. Zhao, R. Ji, Y. Gao, GVCNN: group-view convolutional neural networks for 3D shape recognition, in: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 264–272, doi:[10.1109/CVPR.2018.00035](https://doi.org/10.1109/CVPR.2018.00035).
- [15] A.-A. Liu, W.-Z. Nie, Y. Gao, Y.-T. Su, View-based 3-D model retrieval: a benchmark, *IEEE Trans. Cybern.* 48 (3) (2018) 916–928, doi:[10.1109/TCYB.2017.2664503](https://doi.org/10.1109/TCYB.2017.2664503).
- [16] Z. Han, M. Shang, Z. Liu, C.-M. Vong, Y.-S. Liu, M. Zwicker, J. Han, C.L.P. Chen, SeqViews2SeqLabels: learning 3D global features via aggregating sequential views by RNN with attention, *IEEE Trans. Image Process.* 28 (2) (2019) 658–672, doi:[10.1109/TIP.2018.2868426](https://doi.org/10.1109/TIP.2018.2868426).
- [17] A. Kanezaki, Y. Matsushita, Y. Nishida, RotationNet for joint object categorization and unsupervised pose estimation from multi-view images, *IEEE Trans. Pattern Anal. Mach. Intell.* 43 (1) (2021) 269–283, doi:[10.1109/TPAMI.2019.2922640](https://doi.org/10.1109/TPAMI.2019.2922640).
- [18] X. Wei, R. Yu, J. Sun, View-GCN: view-based graph convolutional network for 3D shape analysis, in: *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 1847–1856, doi:[10.1109/CVPR42600.2020.00192](https://doi.org/10.1109/CVPR42600.2020.00192).
- [19] W. Nie, Y. Zhao, D. Song, Y. Gao, DAN: deep-attention network for 3D shape recognition, *IEEE Trans. Image Process.* 30 (2021) 4371–4383, doi:[10.1109/TIP.2021.3071687](https://doi.org/10.1109/TIP.2021.3071687).

- [20] C. Ma, Y. Guo, J. Yang, W. An, Learning multi-view representation with LSTM for 3-D shape recognition and retrieval, *IEEE Trans. Multimedia* 21 (5) (2019) 1169–1182, doi:[10.1109/TMM.2018.2875512](https://doi.org/10.1109/TMM.2018.2875512).
- [21] Z. Han, H. Lu, Z. Liu, C.-M. Vong, Y.-S. Liu, M. Zwicker, J. Han, C.L.P. Chen, 3D2SeqViews: aggregating sequential views for 3D global feature learning by CNN with hierarchical attention aggregation, *IEEE Trans. Image Process.* 28 (8) (2019) 3986–3999, doi:[10.1109/TIP.2019.2904460](https://doi.org/10.1109/TIP.2019.2904460).
- [22] W. Chu, M. Pelillo, K. Siddiqi, Dominant set clustering and pooling for multi-view 3D object recognition, in: *British Machine Vision Conference*, 2017.
- [23] Y. Guo, F. Sohel, M. Bennamoun, M. Lu, J. Wan, Rotational projection statistics for 3D local surface description and object recognition, *Int. J. Comput. Vis.* 105 (2013) 63–86, doi:[10.1007/s11263-013-0627-y](https://doi.org/10.1007/s11263-013-0627-y).
- [24] Y. Zhang, C. Li, B. Guo, C. Guo, S. Zhang, KDD: a kernel density based descriptor for 3D point clouds, *Pattern Recognit.* 111 (2021) 107691, doi:[10.1016/j.patcog.2020.107691](https://doi.org/10.1016/j.patcog.2020.107691).
- [25] T. Yu, J. Meng, J. Yuan, Multi-view harmonized bilinear network for 3D object recognition, in: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018, pp. 186–194, doi:[10.1109/CVPR.2018.00027](https://doi.org/10.1109/CVPR.2018.00027).
- [26] X. He, T. Huang, S. Bai, X. Bai, View N-Gram network for 3D object retrieval, in: 2019 IEEE/CVF International Conference on Computer Vision (ICCV), 2019, pp. 7514–7523, doi:[10.1109/ICCV.2019.00761](https://doi.org/10.1109/ICCV.2019.00761).
- [27] H. Liu, Y. Cong, C. Yang, Y. Tang, Efficient 3D object recognition via geometric information preservation, *Pattern Recognit.* 92 (2019) 135–145, doi:[10.1016/j.patcog.2019.03.025](https://doi.org/10.1016/j.patcog.2019.03.025).
- [28] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778, doi:[10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90).
- [29] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, ImageNet: a large-scale hierarchical image database, in: 2009 IEEE Conference on Computer Vision and Pattern Recognition, 2009, pp. 248–255, doi:[10.1109/CVPR.2009.5206848](https://doi.org/10.1109/CVPR.2009.5206848).
- [30] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet classification with deep convolutional neural networks, *Commun. ACM* 60 (6) (2017) 84–90, doi:[10.1145/3065386](https://doi.org/10.1145/3065386).
- [31] T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, M. Li, Bag of tricks for image classification with convolutional neural networks, in: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019, pp. 558–567, doi:[10.1109/CVPR.2019.00065](https://doi.org/10.1109/CVPR.2019.00065).
- [32] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, H. Wu, Mixed precision training, in: *International Conference on Learning Representations*, 2018.
- [33] M. Savva, F. Yu, H. Su, A. Kanezaki, T. Furuya, R. Ohbuchi, Z. Zhou, R. Yu, S. Bai, X. Bai, et al., Large-scale 3D shape retrieval from ShapeNet Core55: SHREC'17 track, in: *Proceedings of the Workshop on 3D Object Retrieval*, 2017, pp. 39–50.
- [34] K. Lai, L. Bo, X. Ren, D. Fox, A large-scale hierarchical multi-view RGB-D object dataset, in: 2011 IEEE International Conference on Robotics and Automation, 2011, pp. 1817–1824, doi:[10.1109/ICRA.2011.5980382](https://doi.org/10.1109/ICRA.2011.5980382).
- [35] C. Esteves, Y. Xu, C. Allec-Blanchette, K. Daniilidis, Equivariant multi-view networks, in: 2019 IEEE/CVF International Conference on Computer Vision (ICCV), 2019, pp. 1568–1577, doi:[10.1109/ICCV.2019.00165](https://doi.org/10.1109/ICCV.2019.00165).
- [36] S. Bai, X. Bai, Z. Zhou, Z. Zhang, Q. Tian, L.J. Latecki, Gift: towards scalable 3D shape retrieval, *IEEE Trans. Multimedia* 19 (6) (2017) 1257–1271, doi:[10.1109/TMM.2017.2652071](https://doi.org/10.1109/TMM.2017.2652071).
- [37] S. Bai, X. Bai, Z. Zhou, Z. Zhang, L.J. Latecki, Gift: a real-time and scalable 3D shape search engine, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 5023–5032, doi:[10.1109/CVPR.2016.543](https://doi.org/10.1109/CVPR.2016.543).
- [38] U. Asif, M. Bennamoun, F.A. Sohel, A multi-modal, discriminative and spatially invariant CNN for RGB-D object labeling, *IEEE Trans. Pattern Anal. Mach. Intell.* 40 (9) (2018) 2051–2065, doi:[10.1109/TPAMI.2017.2747134](https://doi.org/10.1109/TPAMI.2017.2747134).
- [39] Y. Cheng, R. Cai, X. Zhao, K. Huang, Convolutional fisher kernels for RGB-D object recognition, in: 2015 International Conference on 3D Vision, 2015, pp. 135–143, doi:[10.1109/3DV.2015.23](https://doi.org/10.1109/3DV.2015.23).
- [40] M.M. Rahman, Y. Tan, J. Xue, K. Lu, RGB-D object recognition with multimodal deep convolutional neural networks, in: 2017 IEEE International Conference on Multimedia and Expo (ICME), 2017, pp. 991–996, doi:[10.1109/ICME.2017.8019538](https://doi.org/10.1109/ICME.2017.8019538).